ClinicQueue — Phase 1 Comprehensive Architectural Guide

A Layman-to-Engineer Walkthrough of the Project Foundation, Structure & Design Decisions

Author: Vaibhav Puvaladas & Antigravity IDE | **Repository:** https://github.com/vaibhavpuvaladas-byte/ClinicQueue | **Commit:** e53c26d | **Milestone:** Phase 1 Complete

1. Executive Summary: What is ClinicQueue & What Did We Do in Phase 1?

🏥 **The Real-World Hospital Analogy:** Imagine entering a bustling hospital outpatient department. If there were no reception desk, no tokens, and no orderly queue, dozens of patients would crowd outside doctors' consultation rooms. Patients would argue over who came first, doctors would be constantly interrupted, and receptionists would be overwhelmed answering "*How long will it take?*".

ClinicQueue is an offline, resilient desktop software system built to solve this exact chaos:

- It gives the receptionist a fast intake screen to register patients and immediately hand them a unique daily token (e.g. `D01-2026-10-20-001`).
- It gives each doctor a dedicated queue board to call the next waiting patient in exact arrival order (First-In, First-Out).
- It gives clinic managers an overview dashboard displaying waiting times, completed visits, and active consultations.

What is "Phase 1"?: If you think of building a house, you cannot install chandeliers, water pipes, or kitchen appliances until you have cleared the land, poured the concrete foundation, erected the steel framework, and installed the doors and windows. **Phase 1 is that rock-solid architectural skeleton.** We created the project blueprint, configured the build tools, built the entire window navigation system, and styled the visual appearance so the application can run smoothly on any computer.

2. The Anatomy of the Codebase: Complete Folder-by-Folder Guide

Every directory in the `ClinicQueue` project was deliberately placed according to industry-standard **Separation of Concerns**. Here is what every single folder does, why it is relevant, and why it is critical:

Directory Path	What Goes Inside?	Why is it Critical? (Layman & Technical Rationale)
/ (Project Root)	Master build files (`pom.xml`), launch wrappers (`mvnw`, `mvnw.cmd`), `.gitignore`, and `README.md`.	The command center of the project. It tells the computer how to compile the code, what libraries to download, and provides first-time setup instructions.
`docs/`	Architectural specifications, live demonstration scripts, and acceptance test plans.	**The Blueprint.** Professional software requires verified documentation so any engineer or stakeholder can understand the design rules without reading thousands of lines of code.
`src/main/java/`	All production Java source code files.	The standard Maven location where the compiler searches for executable application code.
`.../clinicqueue/view/`	Visual user interface classes (`DashboardView`, `RegisterVisitView`, `QueueBoardView`).	**The Display Department.** Contains only what the user sees on their monitor. By isolating UI layouts in their own folder, visual redesigns never risk breaking core business rules or database storage.
`.../clinicqueue/controller/`	UI event handlers and action coordinators.	**The Traffic Police.** When a user clicks "Register" or "Call Next", controllers take that click, validate it, pass it to the business service, and update the screen.
`.../clinicqueue/service/`	Business logic rules (wait time calculator, token allocator, state transition guards).	**The Policy Enforcers.** This folder holds the hospital's operational brain. It ensures a doctor cannot call two patients at once, and guarantees sequential tokens.
`.../clinicqueue/`	Pure JDBC database access classes	**The Filing Clerks.** The only part

repository/	for Patients, Visits, Doctors, and Departments.	of the application permitted to speak SQL to the database. This prevents database queries from being scattered across UI screens.
.../clinicqueue/database/	SQLite connection manager, schema migrations, and table initializers.	**The Secure Vault.** Ensures SQLite foreign keys are always turned on (`PRAGMA foreign_keys = ON;`) and that tables are safely created when the app launches.
.../clinicqueue/model/	Data structures and entity objects (Doctor, Department, Patient, Visit, Status Enums).	**The Standardized Forms.** Represents the medical entities in clean Java records/classes across all layers.
src/main/resources/	Static assets, fonts, icons, and CSS stylesheets.	Separates visual design assets from compiled Java code.
.../css/	Application stylesheet (`app.css`).	**The Interior Designer.** Defines the clinic's color palette, button hover effects, font typography, and status chips in one centralized stylesheet.
src/test/java/	Automated JUnit 5 test classes.	**The Quality Inspector.** Houses automated tests that verify queue logic and token numbering without needing to manually click through the UI.
target/ (Auto-generated)	Compiled `.class` files and packaged JAR binaries.	Created automatically by Maven during compilation. Kept out of Git via `.gitignore` to keep the repository lightweight.

3. Detailed Breakdown of Every File Created in Phase 1

1. `pom.xml` (Project Object Model)

What it is: The recipe and bill-of-materials for the entire application. It instructs Apache Maven on how to assemble ClinicQueue.

- **Java 21 LTS Target:** Configured `<release>21</release>` to utilize the latest Long-Term Support Java features, enhanced pattern matching, and superior runtime performance.
- **JavaFX 21 (Controls & FXML):** Pulls the official OpenJFX desktop graphical engine so modern UI components (buttons, tables, labels) render natively on Windows, macOS, and Linux.
- **SQLite JDBC (3.49.1):** Bundles the lightweight SQLite database driver so ClinicQueue runs as a self-contained desktop app with zero server installation.
- **JUnit 5 (Jupiter):** Modern unit testing framework configured to run during automated builds.

2. `mvnw.cmd` and `mvnw` (Maven Universal Wrappers)

What they are: Intelligent shell scripts (one for Windows `.cmd`, one for Linux/macOS `.sh`) that enable any developer to launch or build ClinicQueue immediately using `.\mvnw.cmd clean javafx:run`, even if they don't have Maven installed globally on their system.

3. `.gitignore`

What it is: A security and hygiene filter for Git. It explicitly forbids compiled bytecodes (`target/`), local SQLite database files (`*.db`), and personal IDE cache (`.idea/`, `.vscode/`) from being accidentally uploaded to GitHub.

4. `Main.java` & `ClinicQueueApp.java` (The Application Core)

How they work together:

- **`Main.java`:** A lightweight bootstrap launcher. In modern Java, launching a JavaFX application from a class that directly extends `Application` can trigger a module-path warning on non-modular classpaths. `Main.java` calls `ClinicQueueApp.main(args)`, completely bypassing this JVM quirk.
- **`ClinicQueueApp.java`:** The primary JavaFX shell. It creates the main desktop window (1280×750 pixels), loads `app.css`, creates the top branding banner, sets today's date, and builds the top navigation bar.
- **The `StackPane` View Switcher:** Instead of opening multiple clunky popups, `ClinicQueueApp` holds `DashboardView`, `RegisterVisitView`, and `QueueBoardView` inside a single central stack. Clicking navigation buttons simply toggles visibility, delivering instant, zero-flicker transitions.

5. `DashboardView.java` (The Control Tower)

Renders high-level clinic metrics: four distinct KPI stat cards (**WAITING**, **IN CONSULTATION**, **COMPLETED**, **CANCELLED**) and a container for per-doctor workload summaries. In Phase 1, it establishes the visual layout ready to receive live database counts in Phase 5.

6. `RegisterVisitView.java` (The Intake Counter)

Provides the patient registration form: Patient Full Name (required), Phone (optional), Department selector, Doctor selector (filtered by department), and Visit Type radio buttons (Walk-in vs Scheduled Appointment).

7. `QueueBoardView.java` (The Doctor's Cockpit)

Provides the doctor's queue management workspace: Doctor selection dropdown, Status filter, search box, "Call Next Patient" primary action button, "Complete Consultation", "Cancel Visit", and the prominent *Approximate Wait Time* banner.

8. `app.css` (Clinical Design System)

Eliminates generic default browser styling in favor of an intentional, clinical palette:

- **Deep Navy Header (`#0f172a`):** Conveys authority, professionalism, and calmness.
- **Vibrant Primary Blue (`#2563eb`):** Highlights active navigation and primary action buttons.
- **High-Contrast Status Chips:** Amber for Waiting (`#fef3c7`), Blue for In Consultation (`#dbeafe`), Emerald Green for Completed (`#dcfce7`), and Slate for Cancelled (`#f1f5f9`).
- **Neutral Card Elevations:** Clean white cards on a soft off-white background (`#f8fafc`) for maximum readability during classroom presentations.

4. Layman's Answers to Key Engineering Questions

Q: Why build a Desktop App with JavaFX instead of a Web Application in React/Chrome? **Answer:** In a real hospital or outpatient clinic, front-desk intake terminals and doctor workstations must work 100% reliably even if the hospital internet drops. A local JavaFX desktop application starts in milliseconds, talks directly to the local database, has zero network latency, and requires no web servers or cloud subscriptions.

Q: Why use Code-Built JavaFX views instead of FXML drag-and-drop files? **Answer:** FXML uses XML text files that look up controllers at runtime using reflection. A single typo in an XML file can crash the app with a confusing error during a live presentation. Code-built Java UI is checked by the Java compiler at compile time—if something is wrong, it won't even compile, guaranteeing stability.

Q: Why SQLite instead of MySQL or MongoDB? **Answer:** SQLite lives in a single local file on disk (`clinicqueue.db`). It requires zero server setup, zero passwords, zero port configurations, and supports full ACID relational transactions and foreign keys. It is the gold standard for reliable, single-computer desktop software.

Q: Why did we separate Views, Services, and Repositories? **Answer:** Imagine a restaurant. The waiter (View) takes your order. The chef (Service) cooks the meal following recipes and safety rules. The pantry manager (Repository) fetches the ingredients from the storeroom. If the waiter tries to cook or rummage through the storeroom, the kitchen devolves into chaos. Separating Views, Services, and Repositories ensures each part of our code does only one job exceptionally well.

5. Phase 1 Verification & Quality Audit

Verification Check	Method Used	Result
Source Code Compilation	Executed `.\mvnw.cmd clean compile` targeting Java 21 release	**BUILD SUCCESS** (11.7s, 0 errors, 5 source files compiled)
Desktop Window Launch	Executed `.\mvnw.cmd javafx:run`	**PASSED** (Main window rendered cleanly at 1280x750)
Version Control Tracking	Pushed to GitHub: `vaibhavpuvaladas-byte/ClinicQueue`	**COMMITTED & PUSHED** (Commit `e53c26d`)

6. What's Next in Phase 2: Persistence & Database Layer

With the skeleton, UI layout, and navigation fully operational, **Phase 2** will connect the plumbing:

1. Establishing the SQLite connection manager with mandatory `PRAGMA foreign_keys = ON;`.
2. Creating database tables for `departments`, `doctors`, `patients`, and `visits`.
3. Implementing pure JDBC repositories using prepared statements (no raw string SQL concatenation).
4. Pre-seeding realistic clinic departments (e.g. Cardiology, Pediatrics) and doctors for immediate demonstration.